Vue.js — Aula 6: Consumindo APIs

Curso de Front-End — Módulo Vue.js

Resumo da aula

Por que usar uma API? Até agora os dados ficavam fixos no `data()`. Com uma API, os dados vêm de um servidor, podem mudar sem editar o código e podem ser diferentes para cada usuário.

fetch() é uma função nativa do JavaScript (não é do Vue) que pede dados a um endereço da web e devolve uma Promise.

```
fetch("https://jsonplaceholder.typicode.com/todos")
  .then((resposta) => resposta.json())
  .then((dados) => console.log(dados))
  .catch((erro) => console.log("deu erro:", erro));
```

Promise é a promessa de um valor que ainda vai chegar. Ela passa por três estados: `pending` (esperando), `resolved` (deu certo, cai no `.then()`) e `rejected` (deu erro, cai no `.catch()`).

async/await faz o mesmo que o `.then()`, mas com código mais parecido com o síncrono. `async` marca a função e `await` espera a Promise resolver.

```
async buscarTarefas() {
  const resposta = await fetch("https://jsonplaceholder.typicode.com/todos");
  this.tarefas = await resposta.json();
}
```

Carregando e erro. Como a requisição demora, usamos os estados `carregando: true` e `erro: null` para o template decidir o que mostrar.

try/catch/finally. O `try` tem o código que pode falhar, o `catch` roda só se der erro (avise o usuário aqui) e o `finally` roda sempre (ótimo para desligar o `carregando`).

```
async created() {
  try {
    const resposta = await
fetch("https://jsonplaceholder.typicode.com/todos");
    this.tarefas = await resposta.json();
  } catch (erro) {
    this.erro = "Não deu para carregar";
  } finally {
    this.carregando = false;
  }
}
```

Exercícios práticos

API usada nos exercícios: `https://jsonplaceholder.typicode.com` (rotas `/todos`, `/users` e `/posts`).

Exercício 1 — Primeiro fetch

Abra o console do navegador e use `fetch()` com `.then()` para buscar `https://jsonplaceholder.typicode.com/todos/1`. Mostre o resultado com `console.log`. Quais campos o objeto tem?

Exercício 2 — Provocando um erro

Repita o exercício 1, mas com um endereço que não existe (por exemplo, `https://servidor-que-nao-existe.xyz/dados`). Use `.catch()` para mostrar "deu erro" no console. Qual Promise estado foi disparado?

Exercício 3 — Trocando por async/await

Reescreva o exercício 1 dentro de uma função `async` usando `await`, sem nenhum `.then()`.

Exercício 4 — Dados na tela

Crie um componente com `data()` contendo `tarefas: []`. Busque `/todos` em um `created()` assíncrono e guarde o resultado em `this.tarefas`. Mostre a quantidade total de tarefas recebidas no template.

Exercício 5 — Lista com v-for

Exiba apenas as 10 primeiras tarefas em uma `<ul>` usando `v-for` com `:key`. Dica: `tarefas.slice(0, 10)`.

Exercício 6 — Estado carregando

Adicione `carregando: true` no `data()`. Mostre "Carregando tarefas..." com `v-if` enquanto os dados não chegam e a lista com `v-else`. Desligue o `carregando` quando a busca terminar.

Exercício 7 — Tratando erros

Adicione `erro: null` no `data()` e envolva a busca com `try/catch/finally`. Se falhar, mostre uma mensagem amigável na tela com `v-else-if`. Teste trocando a URL por uma errada e depois volte para a correta.

Exercício 8 — Tarefas concluídas

Mostre cada tarefa com um ✅ se `t.completed` for `true` e ❌ se for `false`. Em seguida, adicione acima da lista o texto "X de 10 concluídas" usando uma propriedade `computed`.

Exercício 9 — Usuários

Crie uma tela `Usuarios.vue` que busca `/users` e mostra nome, e-mail e cidade (`user.address.city`) de cada usuário, com os estados de carregando e erro. Adicione a rota no Vue Router da Aula 5 e um `router-link` no menu.

Exercício 10 — Desafio: botão recarregar

Transforme a busca em um método `buscarTarefas()` (com `async/await` e `try/catch/finally`). Chame-o no `created()` e também em um botão "Recarregar". Antes de cada nova busca, lembre-se de reiniciar `carregando` e `erro`. Extra: faça o botão ficar desabilitado enquanto carrega (`:disabled="carregando"`).

Recapitulando

Conceito	O que faz
`fetch()`	Faz uma requisição HTTP e devolve uma Promise
Promise	Valor que ainda vai chegar: resolved ou rejected
`async/await`	Escreve código assíncrono no estilo síncrono

Conceito	O que faz
`try/catch/finally`	Trata erros sem quebrar a aplicação
`carregando` / `erro`	Estados que guiam o que o template mostra

Próxima aula: Pinia (estado compartilhado) e projeto final.